

CS-338 Intro to the Theory of Computation

- Lecture #8.1

➤ **Turing Machines**

- Prof Pietro S. Oliveto
Department of Computer Science and Engineering
Southern University of Science and Technology (SUSTech)
- `olivetop@sustech.edu.cn`
<https://faculty.sustech.edu.cn/olivetop>
- Office: Room 113

Overview

Last time:

- CFGs and PDA equivalence
- Non Context Free Languages

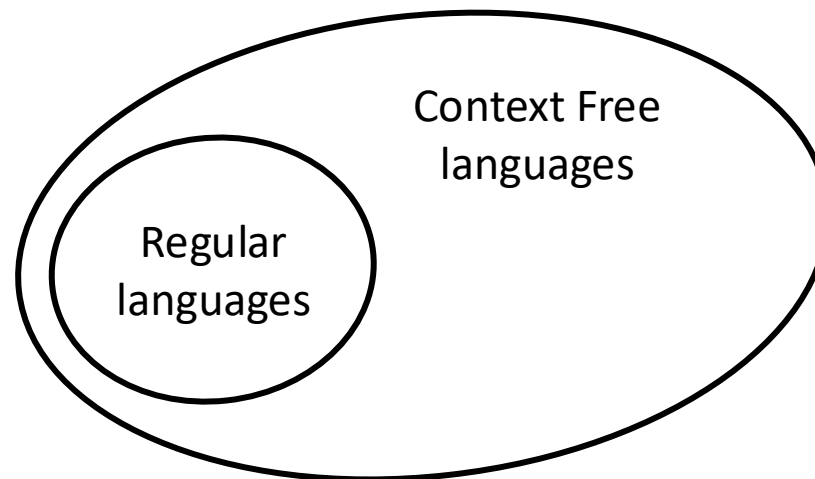
Today: (Sipser §2.3, §3.1)

- Turing machines

Recap

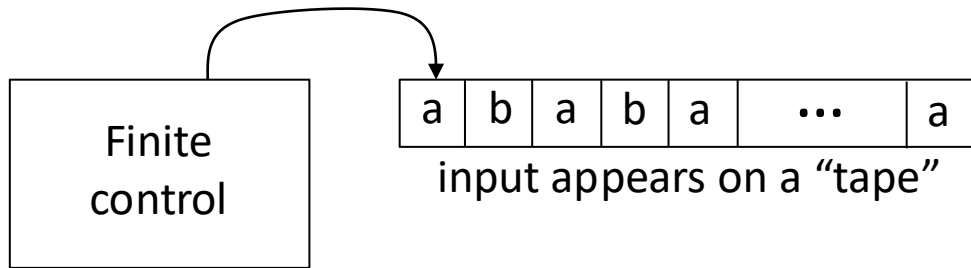
Corollary: Every regular language is context free.

	Recognizer	Generator
Regular language	DFA or NFA	Regular expression
Context Free language	PDA	Context Free Grammar

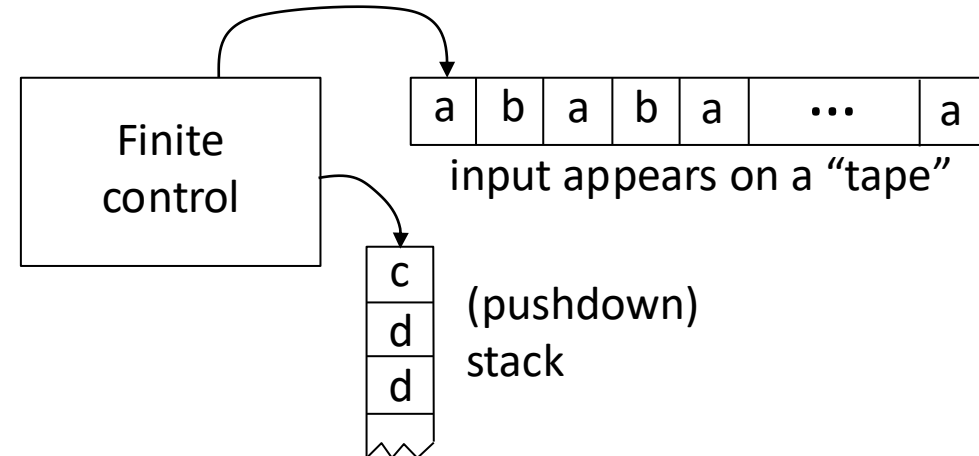


Computability Theory: roadmap

- 1) Finite state automata and regular languages ✓
- 2) (Non-deterministic) Pushdown automata and context free languages ✓
- 3) Turing machines

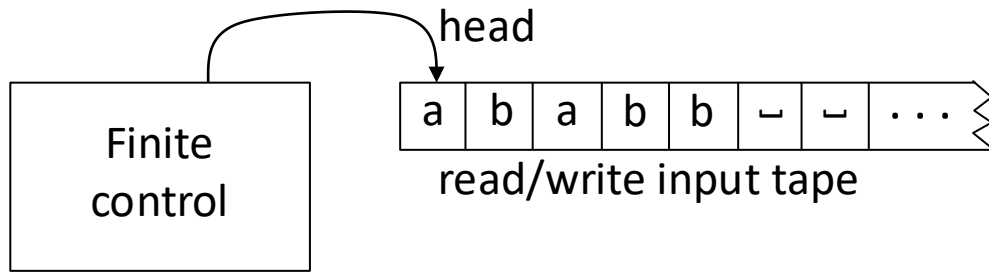


Schematic diagram for DFA



Schematic diagram for PDA

Turing Machines (TMs)



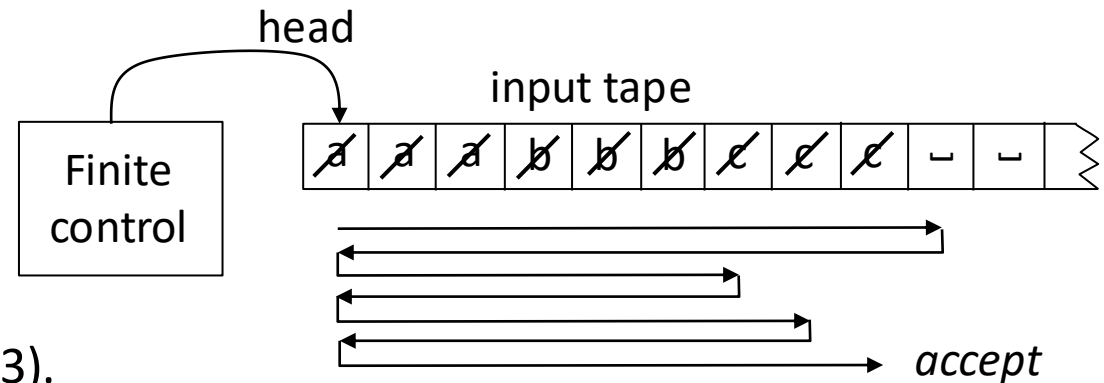
- 1) Head can read and write
- 2) Head is two way (can move left or right)
- 3) Tape is infinite (to the right)
- 4) Infinitely many blanks “ $\sqcup$ ” follow input
- 5) Can accept or reject any time (not only at end of input)

Essentially a finite state automaton with infinite tape that may read and write data

TM – example

TM recognizing $B = \{a^k b^k c^k \mid k \geq 0\}$

- 1) Scan right until $\sqcup$ while checking if input is in $a^*b^*c^*$, *reject* if not.
- 2) Return head to left end.
- 3) Scan right, crossing off single a, b, and c.
- 4) If the last one of each symbol, *accept*.
- 5) If the last one of some symbol but not others, *reject*.
- 6) If all symbols remain, return to left end and repeat from (3).



Check-in 7.2

How do we get the effect of “crossing off” with a Turing machine?

- a) We add that feature to the model.
- b) We use a tape alphabet $\Gamma = \{a, b, c, \cancel{a}, \cancel{b}, \cancel{c}, \sqcup\}$.
- c) All Turing machines come with an eraser.

TM – Formal Definition

Defn:

A *Turing machine* is a 7-tuple, $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, where Q, Σ, Γ are all finite sets and

1. Q is the set of states,
2. Σ is the input alphabet not containing the *blank symbol* $\sqcup$,
3. Γ is the tape alphabet, where $\sqcup \in \Gamma$ and $\Sigma \subseteq \Gamma$,
4. $\delta: Q \times \Gamma \longrightarrow Q \times \Gamma \times \{L, R\}$ is the transition function, (L = Left, R = Right) $\delta(q, a) = (r, b, R)$
5. $q_0 \in Q$ is the start state,
6. $q_{\text{accept}} \in Q$ is the accept state, and
7. $q_{\text{reject}} \in Q$ is the reject state, where $q_{\text{reject}} \neq q_{\text{accept}}$.

From state q reading a the machine writes b replacing a , goes to state r and moves the head to the *right* **after writing**

On input $w \in \Sigma^*$ a TM M may halt (enter q_{acc} or q_{rej}) or M may run forever (“loop”).

So M has 3 possible outcomes for each input w :

1. Accept w (enter q_{acc})
2. Reject w by halting (enter q_{rej})
3. Looping (running forever)

Check-in 7.3

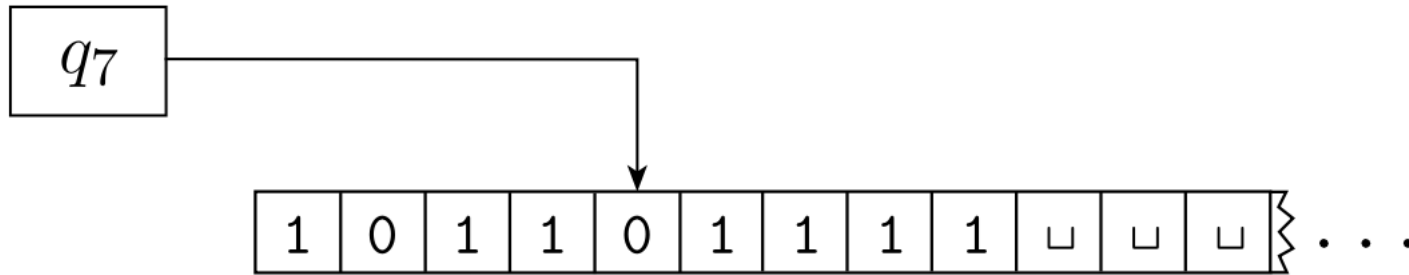
This Turing machine model is deterministic.
How would we change it to be nondeterministic?

- a) Add a second transition function.
- b) Change δ to be $\delta: Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$
- c) Change the tape alphabet Γ to be infinite.

TM – Configurations

- As a TM computes changes occur in current: 1) state; 2) tape contents; 3) head location
- A setting of these three items is called a **configuration** of the TM
- A configuration uqv : means that the current tape contents are uv , the current state is q and the head of the TM is on the first symbol in v .

- Example:



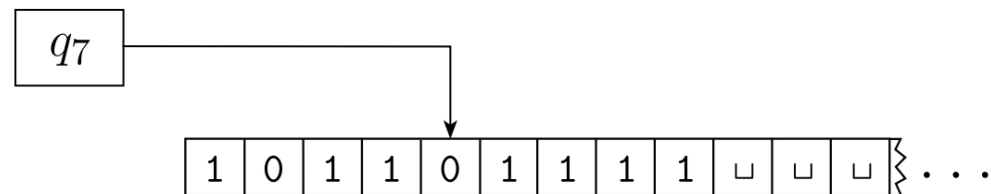
configuration: $1011q_701111$

Start configuration: q_0w

- The input $w = w_1, w_2, \dots, w_n \in \Sigma^*$ is on the first n squares of the tape
- The rest of the tape consists of blank symbols (end of the input as $\sqcup$ does not belong to Σ)

TM – Computation

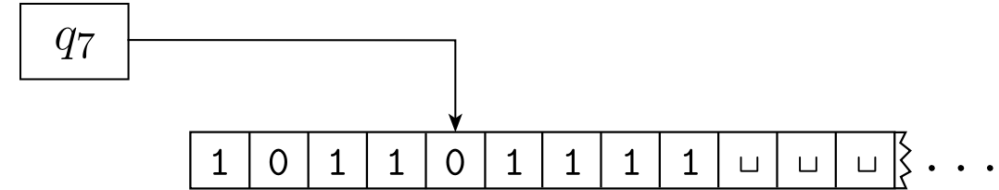
- A configuration **uqv** : means that the current tape contents are uv , the current state is q and the head of the TM is on the first symbol in v .



- A configuration C_1 **yields** configuration C_2 if the TM can legally go from C_1 to C_2 in a single step
- Let $a, b, c \in \Gamma$, $u, v \in \Gamma^*$ and $q_i, q_j \in Q$. Then
 - Left move: $uaq_i b v$ yields $uq_j a c v$ if $\delta(q_i, b) = (q_j, c, L)$
 - Right move: $uaq_i b v$ yields $uacq_j v$ if $\delta(q_i, b) = (q_j, c, R)$
- Special cases
 - Left-hand end: if $\delta(q_i, b) = (q_j, c, L)$, then $q_i b v$ yields $q_j c v$
 - Right-hand end: uaq_i (is equivalent to $uaq_i \sqcup$) and can be handled as before.
- Start configuration**: $q_0 w$
- Halting configurations** are all those with state q_{accept} (**accepting config.**) and q_{reject} (**rejecting config.**)

TM – Computation (2)

- A configuration uqv : means that the current tape contents are uv , the current state is q and the head of the TM is on the first symbol in v .



- A Turing Machine **accepts** input w if a sequence of configurations $C_1, C_2, \dots, C_k$ exist, where:
 - C_1 is the start configuration of M on input w
 - Each C_i yields C_{i+1} , and
 - C_k is an accepting configuration.
- The collection of strings that M accepts is the **language** of M , or the **language recognized** by M , or $L(M)$.

TM Recognizers and Deciders

- Let M be a TM. Then $L(M) = \{w \mid M \text{ accepts } w\}$.

We say that the collection of strings that M accepts is the language of M , or the language recognized by M .

- Say that M recognizes A if $A = L(M)$.

Defn: A is Turing-recognizable if $A = L(M)$ for some TM M .

A TM may accept, reject, or loop. Distinguishing looping for accept can be difficult

Defn: TM M is a decider if M halts on all inputs.

Say that M decides A if $A = L(M)$ and M is a decider.

Defn: A is Turing-decidable if $A = L(M)$ for some TM decider M .

TM Decider – example

TM deciding $B = \{0^{2^n} \mid n \geq 0\}$

Language of all strings with a power of 2 number of 0s

Algorithm?

M_2 = “On input string w :

1. Sweep left to right across the tape, crossing off every other 0.
2. If in stage 1 the tape contained a single 0, *accept*
3. If in stage 1 the tape contained more than a single number of 0s was odd, *reject*.
4. Return the head to the left-hand end of the tape.
5. Go to stage 1.”

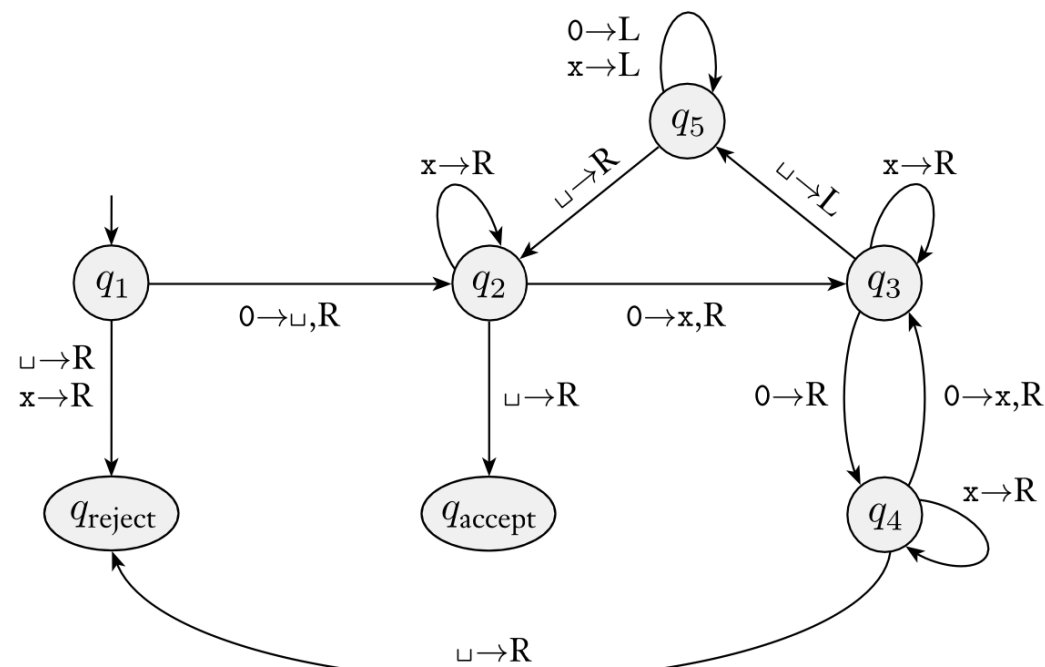
- $Q = \{q_1, q_2, q_3, q_4, q_5, q_{\text{accept}}, q_{\text{reject}}\}$,

- $\Sigma = \{0\}$, and

- $\Gamma = \{0, x, \sqcup\}$.

- We describe δ with a state diagram

- The start, accept, and reject states are q_1 , q_{accept} , and q_{reject} , respectively.



TM – example (2)

TM deciding $B = \{w \# w \mid w \in \{0,1\}^*\}$

Algorithm?

M_1 = “On input string w :

1. Zig-zag across the tape to corresponding positions on either side of the $\#$ symbol to check whether these positions contain the same symbol. If they do not, or if no $\#$ is found, *reject*. Cross off symbols as they are checked to keep track of which symbols correspond.
2. When all symbols to the left of the $\#$ have been crossed off, check for any remaining symbols to the right of the $\#$. If any symbols remain, *reject*; otherwise, *accept*.”

```

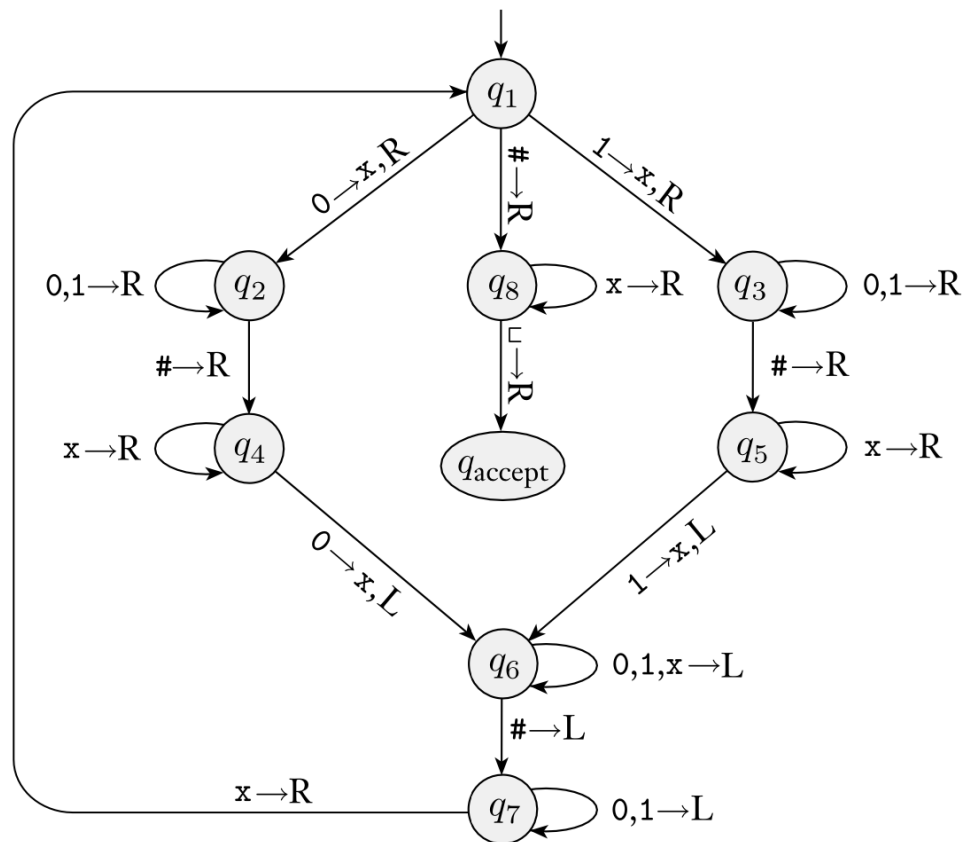
      ↓
0 1 1 0 0 0 # 0 1 1 0 0 0 □ ...
      ↓
x 1 1 0 0 0 # 0 1 1 0 0 0 □ ...
      ↓
x 1 1 0 0 0 # x 1 1 0 0 0 □ ...
      ↓
x 1 1 0 0 0 # x 1 1 0 0 0 □ ...
      ↓
x x 1 0 0 0 # x 1 1 0 0 0 □ ...
      ↓
x x x x x x # x x x x x x □ ...
                                ↓
                                accept
    
```

• $Q = \{q_1, \dots, q_8, q_{\text{accept}}, q_{\text{reject}}\}$,

• $\Sigma = \{0,1,\#\}$, and $\Gamma = \{0,1,\#,x,\sqcup\}$.

• We describe δ with a state diagram (see the following figure).

• The start, accept, and reject states are q_1 , q_{accept} , and q_{reject} , respectively.



TM Decider – example (3)

TM deciding $C = \{a^i b^j c^k \mid i * j = k \text{ and } i, j, k \geq 1\}$

TM decides whether $|a| * |b| = |c|$ (it does some arithmetic)

Algorithm?

$M_3 =$ “On input string w :

1. Scan the input from left to right to determine whether it is a member of $a^+b^+c^+$ and *reject* if it isn't.
2. Return the head to the left-hand end of the tape.
3. Cross off an a and scan to the right until a b occurs. Shuttle between the b 's and the c 's, crossing off one of each until all b 's are gone. If all c 's have been crossed off and some b 's remain, *reject*.
4. Restore the crossed off b 's and repeat stage 3 if there is another a to cross off. If all a 's have been crossed off, determine whether all c 's also have been crossed off. If yes, *accept*; otherwise, *reject*.”

Quick review of today

1. Defined Turing machines (TMs).
2. Defined TM deciders (halt on all inputs).
3. T-recognizable and T-decidable languages.